

State Farm B2B — Developer Guide

Date: July 31, 2026

Table of Contents

State Farm B2B — Developer Guide	3
Quick Reference	3
Architecture Overview	3
Local Development Setup	6
Design System	7
Blocks Reference	8
Common Development Tasks	12
Environments	13
Git Workflow	14
Troubleshooting	14
Resources	14
Support Contacts	15

State Farm B2B — Developer Guide

Quick Reference

Resource	URL
Code Repository	adobedrago/statefarm-b2b-eds (GitHub)
Config Service Org / Site	adobedrago / statefarm-b2b-eds
Preview	https://main-statefarm-b2b-eds-adobedrago.aem.page/
Live	https://main-statefarm-b2b-eds-adobedrago.aem.live/
Local Dev	http://localhost:3000
Content Source	Document Authoring — https://content.da.live/adobedrago/statefarm-b2b-eds

This is a **standard single-repo, single-site** setup: the adobedrago Config Service organization hosts many sites (repoless-style org with 22+ sites), but statefarm-b2b-eds has its own dedicated GitHub repository and content mountpoint — there is no multi-site sharing to worry about for local development.

Architecture Overview

Tech Stack

- Vanilla JavaScript (ES6+), no build step, no transpiling
- CSS3 with Custom Properties, no frameworks
- Content authored in Document Authoring (DA), delivered via Edge Delivery Services
- Linting: ESLint (Airbnb base config) + Stylelint (standard config)

Project Structure

```
├── blocks/           # 12 blocks (10 customized, 2 unmodified boilerplate)
├── scripts/
│   ├── aem.js        # Core library (DO NOT MODIFY)
│   ├── scripts.js    # Custom page decoration, auto-blocking, button/dropdown helpers
│   └── delayed.js     # State Farm chat widget loader
├── styles/
│   ├── styles.css    # Critical styles + design tokens
│   ├── fonts.css     # MecherleSans / Roboto / Roboto Condensed @font-face declarations
│   └── lazy-styles.css # Currently empty placeholder — no non-critical global styles yet
└── icons/           # SVG icons
```

There is no `templates/` folder and no `models/` (Universal Editor component models) folder in this project — it is a pure Document Authoring project, not a Universal Editor / Crosswalk project.

There is also no `scripts/utils.js` — no shared-utility module exists yet. Any cross-block helper logic currently lives duplicated inside individual blocks (see “Notable Implementation Details” in the Blocks Reference below, e.g. the icon-token fallback pattern repeated in `cards`, `cards-service`, and `icon-text`).

Three-Phase Loading (E-L-D)

Edge Delivery Services uses a strict Eager-Lazy-Delayed loading strategy to achieve a Lighthouse score of 100. Every file and script must be placed in the correct phase.

Phase	Purpose	What Loads Here	Performance Impact
Eager	Render above-the-fold content as fast as possible (LCP)	<code>styles/styles.css</code> , first section's blocks, <code>scripts/scripts.js</code> ; <code>fonts.css</code> is loaded eagerly too only if viewport ≥ 900 px or fonts were already cached this session	Blocks LCP — keep minimal
Lazy	Load remaining page content after first paint	Remaining blocks, header (<code>loadHeader</code>), footer (<code>loadFooter</code>), <code>lazy-styles.css</code> , <code>fonts.css</code> (unconditionally, as a fallback for the mobile/first-load case)	Runs after Eager completes
Delayed	Load non-essential third-party scripts	<code>scripts/delayed.js</code> , loaded via <code>window.setTimeout(..., 3000)</code> — currently just the State Farm chat widget	Runs ~3s after page load — never blocks rendering

Rules for developers: - Never add third-party scripts to `scripts.js` — they block LCP. Always use `delayed.js`. - The project already has a conditional font-loading optimization: fonts only load eagerly on desktop-width viewports or once already cached in `sessionStorage` (`fonts-loaded`). Don't remove this without understanding the CLS/LCP tradeoff it was added for. - Blocks in the first section load eagerly; all others load lazily (automatic, based on DOM position via `loadSection/loadSections` from `aem.js`).

A Trusted Types Policy Is Installed at the Top of `scripts.js`

Before anything else runs, `scripts.js` installs a default Trusted Types policy (guarded by a `window.trustedTypes` feature check) that sanitizes **every** HTML sink on the page — `innerHTML`, `outerHTML`, `document.write`, `createContextualFragment`, `srcdoc`, etc. — not just a couple of call sites. This exists because the widget block (see Blocks Reference) assigns fetched remote HTML via plain `.innerHTML`. The policy strips `<script>` tags and removes `srcdoc` from `<iframe>` elements before the HTML is allowed through. **Any new code that sets `innerHTML` with dynamic/remote content is implicitly covered by this policy** — do not bypass it, and be aware that `<script>` tags injected this way will silently vanish rather than execute.

Key Files & How They Connect

File	Role	Connects To
<code>scripts/aem.js</code>	Core library — <code>loadBlock</code> , <code>loadCSS</code> , <code>decorateBlock(s)</code> , <code>decorateIcons</code> , <code>decorateSections</code> , <code>decorateTemplateAndTheme</code> , <code>loadHeader</code> / <code>loadFooter</code> ,	Imported by <code>scripts.js</code> and most blocks

File	Role	Connects To
	sampleRUM, createOptimizedPicture, readBlockConfig, getMetadata, toCamelCase/ toClassName, wrapTextNodes, waitForFirstImage, setup. DO NOT MODIFY.	
scripts/scripts.js	Entry point — installs the Trusted Types policy, orchestrates E-L-D (loadEager → loadLazy → loadDelayed), contains buildAutoBlocks (fragment + widget auto-blocking), decorateButtons, decorateDropdown, decorateMain.	Imports from aem.js; exports decorateMain/ decorateDropdown used by blocks
scripts/delayed.js	Loaded ~3s after page load. Currently: injects the sf-chat custom element script (https://chat.demngd01.c1.statefarm/sf-chat.js) and mounts a <sf-chat> element configured for the b2b experience.	Called by scripts.js via dynamic import() in the delayed phase
styles/styles.css (.light/.highlight, .khaki, .khaki.spacious, .accent-	Critical CSS + design tokens (colors, fonts, spacing) and section styles (heading). Loaded eagerly.	Referenced by all blocks and pages
styles/fonts.css	@font-face declarations for roboto, roboto-condensed (bold/medium/regular weights). Note: the CSS custom properties in styles.css actually reference MecherleSans as the primary body/heading font family — MecherleSans itself is not declared via @font-face in this file (likely loaded elsewhere, e.g. a webfont service, or pending) — verify this before assuming Roboto is what actually renders.	Font families referenced in styles.css custom properties

File	Role	Connects To
styles/lazy-styles.css	Currently just a placeholder comment — no non-critical global styles have been added yet.	Loaded in the lazy phase
blocks/{name}/{name}.js	Block logic — exports default function <code>decorate(block)</code> . Auto-loaded when the block class appears in DOM.	May import from <code>aem.js</code> , <code>scripts.js</code> , or <code>fragment.js</code>
blocks/{name}/{name}.css	Block styles — auto-loaded alongside block JS.	Uses CSS custom properties from <code>styles.css</code>

Execution flow: page load → `scripts.js` installs Trusted Types policy → `loadPage()` → `loadEager()` (decorate main, first section, conditional font load) → `loadLazy()` (remaining sections, header, footer, lazy-styles.css, unconditional font load) → `loadDelayed()` (dynamic import of `delayed.js` after 3s)

Local Development Setup

Prerequisites

- Node.js (no `.nvmrc` or `engines` field is present in this project — use a current LTS Node version)
- AEM CLI: `npm install -g @adobe/aem-cli`

Setup Steps

```
git clone git@github.com:AdobeDrago/statefarm-b2b-eds.git
cd statefarm-b2b-eds

npm install

npx -y @adobe/aem-cli up --no-open --forward-browser-logs
# or, if the CLI is installed globally:
aem up
```

Local Server

- URL: `http://localhost:3000`
- Auto-reload on file changes
- Serves your local (even uncommitted) code, but content must have been **previewed in Document Authoring** at least once to render — DA is the source of truth for content, local `.plain.html` files under `drafts/` are the exception (see below)

Testing Without Authored Content

If no DA-authored page exists yet to test against, create static HTML files under a `drafts/` folder at the project root (this project already has one) and start the CLI with:

```
aem up --html-folder drafts
```

Files must follow the AEM markup structure and use `.html` or `.plain.html` extensions.

Linting

```
npm run lint      # ESLint (airbnb-base) + Stylelint (standard config)
npm run lint:fix  # auto-fix both
```

Key lint rules enforced beyond the defaults (`.eslintrc.js`): - `import/extensions` — all JS imports must include the `.js` extension - `linebreak-style` — Unix (LF) line endings only - `no-param-reassign` — relaxed to allow mutating properties of a parameter (needed for in-place DOM decoration)

Design System

CSS Custom Properties (`styles/styles.css`)

Colors

```
--background-color: white;
--light-color: #f8f8f8;
--dark-color: #505050;
--text-color: #3b3b3b;
--link-color: #d62311;           /* State Farm red */
--link-hover-color: #b81c0e;
--khaki-color: #f7f0e4;         /* used by the "khaki" section variant */
```

Typography

```
--body-font-family: MecherleSans, Arial, sans-serif;
--heading-font-family: MecherleSans, Arial, sans-serif;

/* mobile-first sizes */
--body-font-size-m: 22px;
--body-font-size-s: 19px;
--body-font-size-xs: 17px;
--heading-font-size-xxl: 55px;
--heading-font-size-xl: 44px;
--heading-font-size-l: 34px;
--heading-font-size-m: 27px;
--heading-font-size-s: 24px;
--heading-font-size-xs: 22px;

/* ≥900px overrides (desktop sizes are actually smaller than mobile here – verify intentional) */
--body-font-size-m: 16px;
--body-font-size-s: 16px;
--body-font-size-xs: 14px;
--heading-font-size-xxl: 48px;
--heading-font-size-xl: 36px;
--heading-font-size-l: 28px;
--heading-font-size-m: 28px;
--heading-font-size-s: 20px;
--heading-font-size-xs: 20px;
```

Note: the desktop ($\geq 900\text{px}$) font-size overrides are *smaller* than the mobile defaults for every token. This is unusual for a mobile-first system and worth confirming with design before “fixing” — it may be intentional (e.g. mobile needs larger tap-friendly text) or a leftover from an import.

Fonts (styles/fonts.css)

Family	Weights	Format
roboto-condensed	700 (bold)	woff2
roboto	700 (bold), 500 (medium), 400 (regular)	woff2

All fonts use `font-display: swap` and a Latin-range unicode-range subset. As noted above, MecherleSans — the font actually referenced by the design tokens — has no `@font-face` declaration in this file; confirm where/how it’s actually being loaded before relying on it.

Breakpoints

This project uses the modern CSS range syntax (`width >= 900px`) rather than `min-width`. Only one breakpoint is in active use:

Name	Breakpoint	Usage
Mobile	default (no media query)	Base/mobile-first styles
Desktop	<code>@media (width >= 900px)</code>	Section padding, khaki-spacious padding, font-size overrides

The standard 600px/1200px tier breakpoints mentioned in project conventions are **not currently used** anywhere in `styles.css` — only the 900px tier appears. Block-level CSS may introduce its own breakpoints (e.g. `header.css`, `cards.css`) — check each block’s stylesheet for its own responsive rules.

Section Metadata Styles

Class	Effect
<code>.section.light</code> , <code>.section.highlight</code>	Full-bleed light gray (<code>--light-color</code>) background band, no outer margin, 40px vertical padding
<code>.section.khaki</code>	Framed, rounded, drop-shadowed box (not full-bleed) in the khaki brand color, inset within the normal content width
<code>.section.khaki.spacious</code>	Same as <code>khaki</code> but with 40px padding at $\geq 900\text{px}$ (for boxes with more content, e.g. multiple lists)
<code>.section.accent-heading</code>	Colors the section’s first <code><h3></code> in brand red (<code>--link-color</code>)

Blocks Reference

Of the 12 blocks in `blocks/`, **10 are customized** for this project (documented below) and 2 (`fragment`, `hero`) are unmodified boilerplate carried over from `aem-boilerplate` (only an “Initial commit” in their history) — not documented here since they add no project-specific behavior beyond the stock implementation.

Block	Purpose	DOM Input/Output	Variants	Dependencies	Key Functions	Gotchas
bread-crumbs	Renders an authored link trail as a semantic breadcrumb nav, converting the last item into the current-page marker.	In: single <code></code> of <code><a></code> links. Out: <code><nav aria-label="Breadcrumb">...</nav></code> ; last <code><a></code> replaced with <code></code> .	None.	None.	<code>decorate()</code> — <code>ul</code> → <code>ol</code> swap + current-page span replacement.	Silently no-ops if no <code></code> found. Chevron separator is a CSS border-triangle, not an image.
cards	Generic 2-per-row card grid (image + body) for teaser/promo content.	In: each row → one card; first cell classified image-only vs. body. Out: <code></code> with <code>.cards-card-image/.cards-card-body</code> ; images run through <code>createOptimizedPicture(750w)</code> .	None (1-col mobile, 2-col ≥600px grid).	<code>createOptimizedPicture(aem.js)</code> .	<code>decorate()</code> — reclassifies row divs, rebuilds <code>/</code> .	“Login required” lock icon is authored as a bare trailing <code></code> (not <code>span.icon</code>) because DA strips <code></code> from cells — CSS positions it absolutely.
cards-service	Service/product card grid with pictogram icons and an optional link dropdown per card.	In: icon cell (picture / pre-decorated icon span / <code>:icon:token</code>) + body cell, optionally a nested <code></code> of links. Out: <code></code> cards; nested link list becomes a <code><select></code> via <code>decorateDropdown</code> .	Structural pictogram vs. photo card styling (not an authored class).	<code>createOptimizedPicture</code> , <code>decorateIcons(aem.js)</code> , <code>decorateDropdown(scripts.js)</code> .	<code>decorate()</code> — 3-way icon classification conversion.	Relies on global <code>decorateIcons</code> having run first, but has a fallback for undecorated <code>:token:paragraphs</code> . Dropdown strip pinned to card bottom via <code>margin-top: auto</code> .
columns	Generic responsive multi-column layout, stacked mobile / side-by-side ≥900px.	In: one row of <code><div></code> columns. Out: <code>columns-N-cols</code> class; image-only column gets <code>.columns-img-col</code> .	<code>.quarter</code> — 25/75 image/text split, vertically centered (vs. default even split, top-aligned).	None.	<code>decorate()</code> — column count + image-column flagging.	Image detection requires the picture be the column's sole child. <code>.quarter</code> must be authored as an extra block class.
columns-auth	Login/authenticated hero variant: login	In: <code>h1/copy</code> , a “Log In” link, an <code>h6</code> “No lo-	<code>columns-auth-3-cols</code> triggers	<code>decorateIcons(aem.js)</code> , <code>decorateLoginButtons(aem.js)</code> , <code>decorateNoLoginList(aem.js)</code>	<code>decorateLoginButtons(aem.js)</code> , <code>decorateNoLoginList(aem.js)</code> , <code>decorate()</code> .	NO LOGIN_ICONS is a closed 3-entry exact-

Block	Purpose	DOM Input/Output	Variants	Dependencies	Key Functions	Gotchas
	CTA button + “no login required” quick-link row with mapped icons. comes	gin required.” + <code></code> . Out: <code>columns-auth-N-cols</code> ; “Log In” button. <code>accent</code> ; no-login <code></code> gets icons via a hardcoded label→icon map.	a fixed <code>icon:title:actions</code> ≈ 1:3:2 layout instead of the default 5:7 split.			text map (Request supplement, Create assignment, Fire service provider tool) — a wording change silently drops the icon with no error.
columns-promo	Promotional callout columns with a bordered/cream card look and an optional product-select drop-down.	In: <code>columns-table</code> + a plain <code></code> anywhere in the block. Out: <code>columns-promo-N-cols</code> ; image column flagged; every <code></code> converted to a <code><select></code> .	None beyond column count.	<code>decorateDropdown</code> (scripts.js).	<code>decorate()</code> .	Card capped at 1060px (narrower than the 1200px page width). Converts all <code></code> s to dropdowns, including any plain bullet list not meant as navigation.
footer	Loads the shared footer fragment and splits it into a red slogan banner + cream content area (links/notices/copyright).	In: empty block; fragment resolved from footer page metadata → /content/footer → /footer. Out: <code>.footer-banner</code> , <code>.footer-content</code> with <code>.footer-links</code> / <code>.footer-notices</code> / <code>.footer-copyright</code> , hidden <code>.footer-logo</code> .	None.	<code>getMetadata</code> (aem.js); <code>loadFragment</code> (blocks/fragment/fragment.js).	<code>decorate()</code> (async), 3-path fallback + positional classification.	Silently renders nothing if no fragment resolves. Classification is purely positional (1st <code></code> = links, 2nd = notices, last text-only <code><p></code> = copyright) — re-ordering authored content breaks styling. Logo is <code>display:none</code> , not removed — intentional, not a bug.
header	Loads the shared nav fragment; builds brand logo, megamenu, JS-built search pane, login CTA, mobile	In: empty block; fragment resolved from nav metadata → /content/nav → /nav. First 3 sec-	Responsive only (desktop ≥900px vs. mobile hamburger) — no authored variants.	<code>getMetadata</code> (aem.js); <code>loadFragment</code> (fragment.js).	<code>closeAllDrops/openDrop</code> , <code>toggleMenu</code> , <code>decorateSearch</code> , <code>decorate()</code> .	See “Notable Implementation Details” below for megamenu/search-pane/breakpoint logic. Silently renders an

Block	Purpose	DOM Input/Output	Variants	Dependencies	Key Functions	Gotchas
	hamburger/accordion.	tions map to brand/sections/tools; nav items with a nested <code></code> become megamenu triggers. Out: full <code><nav id="nav"></code> with <code>.nav-brand</code> , <code>.nav-sections</code> (<code>.nav-drop</code> + detached <code>.nav-drop-panels</code>), <code>.nav-tools</code> , JS-built <code>.nav-search-pane</code> , <code>.nav-login-wrap</code> , <code>.nav-hamburger</code> .				empty header if the nav fragment can't be resolved.
icon-text	Simple icon + text row list (e.g. contact methods, feature list).	In: icon cell (picture / pre-decorated icon span / :icon: token) + body cell per row. Out: <code>.icon-text-icon</code> / <code>.icon-text-body</code> .	<code>.small</code> — fixed 56px icon (vs. default 64px mobile/120px desktop), authored as a block-level class (cell classes don't survive DA normalization).	<code>decorateIcons</code> (aem.js).	<code>decorate()</code> .	<code>.small</code> must be on the block, not a cell — common authoring mistake when trying to size one row's icon.
widget	Generic embed mechanism: fetches and injects an external mini-app (HTML/CSS/JS) as if it were a block.	In: block contains one <code></code> . Out: <code>block/wrapper/container</code> classes renamed to <code>{widgetName}/-wrapper/-container</code> ; content replaced with fetched HTML; widget CSS loaded, JS module's	Behavior fully delegated to the linked widget's own JS/CSS — no variants on the widget block itself.	<code>loadCSS</code> (aem.js); <code>dynamic import()</code> of the widget's own module at runtime.	<code>parseWidgetHref()</code> , <code>widgetUrl()</code> , <code>applyWidgetShell()</code> , <code>decorate()</code> .	Path validation blocks <code>./..</code> traversal but otherwise trusts the href fully — broken paths fail only at runtime (caught, console-logged, block left half-decorated). Query params become dataset attrs on the block before the widget's

Block	Purpose	DOM Input/Output	Variants	Dependencies	Key Functions	Gotchas
		default export invoked with the block.				JS runs — the widget's only configuration channel. Fetches 3 separate remote assets per instance, no caching/dedup.

Notable Implementation Details

- **Header megamenu panel positioning:** Dropdown panels are detached from their trigger `<li>` and moved into a single trailing `.nav-drop-panels` container (`position: fixed` on desktop) so top-level trigger links stay ahead of nested panel links in DOM/tab order — important for keyboard navigation. On open, `header.js` reads `nav.getBoundingClientRect().bottom` and sets it as the panel host's `top` inline style on every open (not just once), so it stays correct if the nav bar's height changes.
- **Header search pane vs. megamenu — different overlay strategies:** Megamenu panels are `position: fixed` overlays that don't affect page flow. The search pane instead lives as its own CSS grid row inside `<nav>`; opening it animates that row's track from `0fr` to `1fr`, pushing the rest of the page down. Preserve this distinction when extending the header rather than assuming all header overlays behave the same way.
- **Header breakpoint reset:** A single shared `matchMedia('(min-width: 900px)')` listener (`isDesktop`) calls `toggleMenu()` and `closeAllDrops()` whenever the breakpoint is crossed, so a menu/dropdown left open in one mode doesn't linger broken after a resize.
- **Footer/header shared fragment-loading pattern:** Both blocks use an identical 3-tier fallback for locating their fragment (page metadata $\rightarrow$ `/content/...` $\rightarrow$ `/...`), reflecting a migration where content moved under `/content/` but older paths are still supported. Both fail silently (empty block, no console error) if nothing resolves — a “nothing rendered” symptom with no error signal to chase.
- **columns-auth's hardcoded icon-label map:** `NO_LOGIN_ICONS` is the block's only authoring-driven (non-structural) logic — an exact-text lookup from link label to icon name. A content author's wording tweak silently drops the icon with no visible error.
- **widget as an extension point:** Unlike every other block, `widget.js` doesn't decorate authored markup directly — it's a loader for an independently-versioned mini-app under `/widgets/`, configured via URL query params forwarded as dataset attributes. This is the block most likely to need custom debugging (check Network tab for 404s on the three fetched assets, and console for the caught/logged error).
- **Shared DA cell-content constraint:** `cards`, `cards-service`, and `icon-text` each work around the fact that DA's block-authoring round-trip strips `<span>` tags from cell content — icons are authored either as a bare `<img>/<picture>` or as a `:icon-name:` text token, and each block's JS has its own token-to-span fallback conversion. Worth knowing before writing any new block that needs inline icons in a table cell — consider extracting this into a shared `utils.js` helper rather than re-duplicating it a fourth time.

Common Development Tasks

Add a New Block

1. Create `blocks/{name}/{name}.js` exporting default function `decorate(block)`
2. Create `blocks/{name}/{name}.css`, scoped to `.{name}` (never bare classes like `.item-list`)

3. Avoid the `{blockname}-container/{blockname}-wrapper` class names — those are reserved for auto-generated section wrapper elements
4. Test locally at `http://localhost:3000` (use `drafts/ + --html-folder drafts` if no DA content exists yet)
5. Run `npm run lint` before committing
6. Push to a feature branch and verify on the feature preview URL before opening a PR

Modify Global Styles

1. Edit `styles/styles.css` for critical/design-token changes, `styles/lazy-styles.css` for non-critical global styles
2. Reuse existing CSS custom properties rather than hardcoding new color/spacing values
3. Test across multiple pages/blocks — global changes have wide blast radius
4. Watch for CLS impact, especially around fonts (see the conditional font-loading logic in `scripts.js`)

Add Analytics/Marketing Tooling

1. Add to `scripts/delayed.js`, following the pattern already used for the `sf-chat` widget (create + append DOM nodes, no top-level render-blocking work)
2. Never add to `scripts.js` — it blocks LCP
3. Verify via the Network tab that the resource loads ~3 seconds after page load, not eagerly

Debug a Block

1. Check the browser console — `fragment/nav/footer`-dependent blocks (header, footer) fail silently if their fragment path can't be resolved; the widget block does log a caught error
2. Inspect the DOM: compare the actual decorated structure against the “DOM Output” column in the Blocks Reference table above
3. Check whether an expected variant class landed on the block level vs. a cell level (a recurring gotcha — see `icon-text.small` and `columns-auth's exact-text` link matching)
4. For icon-related bugs, check whether the icon was authored as a picture, a pre-decorated `span.icon`, or a `:icon:` text token — each block's fallback logic differs slightly

Environments

Environment	URL Pattern	Purpose
Local	<code>http://localhost:3000</code>	Development (local code + previewed DA content)
Feature Branch	<code>https://{branch}--statefarm-b2b-eds--adobedrago.aem.page</code>	PR testing
Preview	<code>https://main--statefarm-b2b-eds--adobedrago.aem.page/</code>	Staging (same content as local)
Live	<code>https://main--statefarm-b2b-eds--adobedrago.aem.live/</code>	Production (author-approved content)

Git Workflow

Branch Naming

Existing branches follow short, descriptive kebab-case names (e.g. `fix-columns-promo-mobile`, `icon-text-block`, `icon-text-link-weight-fix`) — keep names short due to URL length limits in feature preview URLs.

PR Requirements

1. Include a feature preview URL (with a specific content path demonstrating the change) in the PR description
2. Pass `npm run lint`
3. Verify the change on the feature branch preview
4. Run a PageSpeed Insights check against the feature preview and target a score of 100
5. Use `gh pr checks` to verify code sync / lint / performance test status before requesting review

Troubleshooting

Block not loading?

1. Check the block folder name matches the authored block/class name
2. Verify the JS exports `default function decorate(block)`
3. Check the Network tab for 404s on the block's JS/CSS

Styles not applying?

1. Check CSS specificity and confirm the selector is scoped to `.{blockname}` (per project convention)
2. Verify the stylesheet is actually loading (Network tab)
3. Run `npm run lint:css` to catch syntax errors

Content not updating?

1. Remember DA is the source of truth — a local `.plain.html` won't render until the corresponding page has been previewed in Document Authoring (except for static files under `drafts/`, tested via `--html-folder drafts`)
2. Confirm the page was actually previewed in DA
3. Allow 1-2 minutes for CDN cache to catch up

Fragment-based block renders empty (header/footer)?

1. Check that a nav/footer fragment exists at one of the three resolved paths (page metadata → `/content/{name}` → `{name}`)
2. These blocks fail silently — no console error — so an empty header/footer with no error is the primary symptom, not a bug indicator by itself

Resources

Resource	URL
Edge Delivery Services Documentation	https://www.aem.live/docs/

Resource	URL
Developer Tutorial	https://www.aem.live/developer/tutorial
Anatomy of a Project	https://www.aem.live/developer/anatomy-of-a-project
Markup, Sections, Blocks	https://www.aem.live/developer/markup-sections-blocks
E-L-D Loading / Web Performance	https://www.aem.live/developer/keeping-it-100
Best Practices	https://www.aem.live/docs/dev-collab-and-good-practices

Support Contacts

Admin access (per Config Service): gallardo@adobe.com, mdean@adobe.com. See `AGENTS.md` in the project root for the full contributor/agent workflow, skill routing table, and publishing process.